

ĐẠI HỌC QUỐC GIA THÀNH PHỐ HỒ CHÍ MINH
TRƯỜNG ĐẠI HỌC CÔNG NGHỆ THÔNG TIN
KHOA KHOA HỌC MÁY TÍNH



BÁO CÁO SEMINAR
MÔN KỸ THUẬT LẬP TRÌNH TRÍ TUỆ NHÂN TẠO
Đề tài: Trợ lý học tập và nghề nghiệp dựa trên CV
(SkilBridge)

GVHDTH: Đặng Văn Thìn

Nhóm sinh viên thực hiện:

1. Võ Phước Thịnh

MSSV: 24521711

2. Liên Phúc Thịnh

MSSV: 24521686

□□ Tp. Hồ Chí Minh, 01/2026 □□

NHẬN XÉT CỦA GIÁO VIÊN HƯỚNG DẪN

....., ngày.....tháng.....năm 20...

Người nhận xét

(Ký tên và ghi rõ họ tên)

BẢNG PHÂN CÔNG, ĐÁNH GIÁ THÀNH VIÊN:

Bảng 1: Bảng phân công, đánh giá thành viên

Họ và tên	MSSV	Phân công	Đánh giá
Võ Phước Thịnh(lead)	24521711	Làm backend(intelligence layer, pipeline) và slide	Hoàn thành tốt trong công việc mà lead đưa ra
Liên Phúc Thịnh	24521686	Làm frontend và backend(logic layer) và deployment	Hoàn thành tốt trong công việc mà lead đưa ra

LỜI MỞ ĐẦU

Đề tài: Xây dựng hệ thống phân tích và tối ưu hóa hồ sơ năng lực ứng viên (AI Resume Analyzer) ứng dụng Large Language Models.

1. Đặt vấn đề:

Trong bối cảnh thị trường tuyển dụng ngày càng cạnh tranh, việc đánh giá thủ công mức độ phù hợp giữa Hồ sơ năng lực (CV) và Mô tả công việc (Job Description - JD) tiêu tốn nhiều thời gian và dễ mang tính chủ quan. Đối với ứng viên, việc thiếu công cụ đánh giá khách quan khiến họ khó nhận ra những "lỗ hổng" kỹ năng cần cải thiện để đáp ứng yêu cầu của nhà tuyển dụng.

Xuất phát từ vấn đề đó, trong khuôn khổ môn học **CS311**, nhóm chúng em (gồm Võ Phước Thịnh và Liên Phúc Thịnh) đã nghiên cứu và phát triển dự án **AI Resume Analyzer v3.0** và là một giải pháp toàn diện hỗ trợ phân tích CV thông minh.

Link Website: <https://ineedjob.thinhopsops.win>

Link Github: <https://github.com/TheChaser-life/CS311>

Link Dataset:  [data CS311](#)

MỤC LỤC

LỜI MỞ ĐẦU

4

Chương 1: Tổng quan đề tài (Introduction)

- 1.1. Đặt vấn đề
- 1.2. Mục tiêu đề tài

Chương 2: Kiến trúc Agent và Cơ chế Xử lý Nâng cao

- 2.1. Cơ chế Lập luận và Điều hướng Tác vụ (ReAct & Tool Routing)
- 2.2. Kỹ thuật Prompt Engineering & Structured Output Parsing
- 2.3. Quản lý Trạng thái Hội thoại
- 2.4. RAG & Dynamic Query Generation

Chương 3: Phân tích và Thiết kế hệ thống (System Analysis & Design)

3.1 Kiến trúc tổng thể (Architecture)

3.2. Quy trình hoạt động của ứng dụng

3.3. Thiết kế các Module nòng cốt và các tools tích hợp kèm theo

Chương 4: Thực nghiệm và đánh giá

4.1. Đánh giá các chức năng của Agent

4.2. Đánh giá hiệu năng hệ thống

Chương 5: Kết luận và Hướng phát triển (Conclusion & Future Work)

5.1. Kết luận

5.2. Hạn chế

5.3. Hướng phát triển (Future Work):

- **Phỏng vấn ảo (Mock Interview)**
- **Thiết kế lộ trình học tập và thực hành cho ứng viên**
- **Gợi ý các sự kiện, kì thi và chứng chỉ ý nghĩa**
- **Hệ thống gợi ý hai chiều**
- **Tối ưu hóa chi phí**
- **Cơ chế tạo tài khoản và đăng nhập**
- **Nâng cao khả năng lưu trữ**
- **Scaling ứng dụng**

Chương 1: Tổng quan đề tài

1.1 Đặt vấn đề

Trong kỷ nguyên số, quy trình tuyển dụng đang đối mặt với những thách thức lớn từ cả hai phía: Nhà tuyển dụng và ứng viên. Sự bùng nổ của thị trường lao động trực tuyến dẫn đến tình trạng quá tải thông tin, khiến việc kết nối đúng người - đúng việc trở nên khó khăn hơn bao giờ hết.

Đối với nhà tuyển dụng:

- **Quá tải hồ sơ:** Mỗi tin đăng tuyển có thể nhận hàng trăm CV, nhưng tỷ lệ ứng viên đạt yêu cầu thường rất thấp. Quy trình sàng lọc thủ công tiêu tốn thời gian và nguồn lực lớn, dễ dẫn đến bỏ sót nhân tài do mệt mỏi hoặc định kiến chủ quan.
- **Hiệu suất thấp:** Thời gian trung bình để xem xét một CV chỉ khoảng 6-7 giây, không đủ để đánh giá sâu năng lực tiềm ẩn của ứng viên.

Đối với ứng viên:

- **Thiếu định hướng:** Nhiều ứng viên, đặc biệt là sinh viên mới ra trường, có thói quen "rải" hồ sơ hàng loạt mà không thực sự hiểu rõ yêu cầu công việc.
- **Rào cản ATS:** CV thường bị loại sớm bởi các hệ thống theo dõi ứng viên do lỗi định dạng hoặc thiếu từ khóa, dù ứng viên có đủ năng lực.
- **Mất phương hướng:** Không biết bản thân thiếu kỹ năng gì so với nhu cầu thị trường để bổ sung kịp thời

1.2 Mục tiêu đề tài

Dựa trên những vấn đề đã được nêu trên, nhóm em đã cùng nhau đề xuất xây dựng hệ thống Smart Resume Analyst 3.0 với các mục tiêu cụ thể sau:

Tự động hóa phân tích & Đánh giá:

- Xây dựng hệ thống có khả năng đọc hiểu đa định dạng bằng công nghệ OCR tích hợp AI.
- Tính toán điểm phù hợp giữa CV và JD dựa trên ngữ nghĩa hay vì chỉ so khớp từ khóa đơn thuần.

Tư vấn & Định hướng nghề nghiệp:

- Phân tích khoảng cách kỹ năng: Chỉ rõ ứng viên đang thiếu kỹ năng gì so với yêu cầu tuyển dụng.
- Đề xuất lộ trình học tập và các khóa học bổ sung phù hợp.

Hỗ trợ tìm kiếm việc làm:

- Tích hợp công cụ tìm kiếm thời gian thực để tự động gợi ý các công việc đang tuyển dụng phù hợp với hồ sơ năng lực của ứng viên.

Nâng cao chất lượng hồ sơ:

- Cung cấp tính năng Chatbot Mentor để tư vấn sửa đổi nội dung và bố cục (Layout) CV sao cho đúng với format với điều kiện của nhà tuyển dụng trong ATS.

Chương 2: Kiến trúc Agent và Cơ chế Xử lý Nâng cao

Trong phiên bản v3.0, hệ thống không hoạt động theo mô hình tuyến tính truyền thống mà chuyển sang kiến trúc Agentic Workflow. Tại đây, mô hình ngôn ngữ lớn đóng vai trò là bộ não điều phối, có khả năng tự suy luận và quyết định công cụ cần sử dụng dựa trên ngữ cảnh.

2.1. Cơ chế Lập luận và Điều hướng Tác vụ (ReAct & Tool Routing)

Cốt lõi trí tuệ của hệ thống nằm ở việc áp dụng mô hình ReAct (Reasoning + Acting), cho phép AI phá vỡ giới hạn của một Chatbot thông thường để trở thành một Agent có khả năng hành động thực tế.

a. Nguyên lý hoạt động của ReAct Loop

Thay vì ánh xạ trực tiếp từ input sang output thì hệ thống áp dụng vòng lặp suy luận ba bước cho mỗi yêu cầu của người dùng:

1. Thought (Suy luận):

Ví dụ điển hình:

Khi nhận được yêu cầu (ví dụ: "Tìm việc Python tại TP.HCM cho tôi"), Agent không trả lời ngay. Trước tiên, nó phân tích ý định (ở đây theo hành động là "tìm", hiểu được "tìm", và sau đó phân tích ngữ cảnh là "việc Python tại TP.HCM", đối với cá nhân "cho tôi") để suy luận và đưa ra kế hoạch cho hành động tiếp theo của Agent.

2. Action (Hành động):

Dựa trên suy luận, Agent quyết định gọi một "Tool"(module, chức năng độc quyền) cụ thể.

Ví dụ: Trong hệ thống này, các Tool được định nghĩa là các hàm Python (`find_jobs`, `analyze_cv`) được đóng gói chuẩn giao tiếp API.

3. Observation:

Agent nhận kết quả trả về từ Tool (ví dụ: danh sách việc làm từ Tavily API) và nạp lại vào ngữ cảnh để tổng hợp câu trả lời cuối cùng.

Ví dụ thực tế:

- **User:** "Hồ sơ của tôi có hợp với vị trí Backend Developer không?"
- **Agent Thought:** User đang muốn đánh giá độ phù hợp => Cần so sánh nội dung CV và JD => Phải gọi tool `analyze_cv`.
- **Agent Action:** Gọi hàm `analyze_cv(cv_text, jd_text)`.

b. Cơ chế Tool Routing (Điều hướng công cụ):

Hệ thống sử dụng kỹ thuật Tool Binding (Tool binding của LangChain để gắn các năng lực cụ thể cho mô hình GPT-4o). Quá trình này diễn ra như sau:

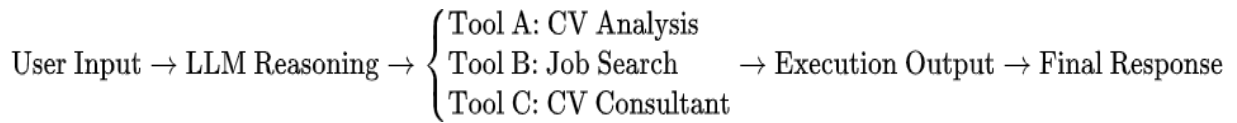
- **Bước 1 - Khai báo Tool:** Các chức năng cốt lõi được định nghĩa dưới dạng `@tool decorator`, kèm theo docstring mô tả kỹ lưỡng chức năng của module. Nói cách khác, docstring này chính là bản hướng dẫn sử dụng để LLM hiểu khi nào nên dùng tool này.
 - Ví dụ: Tool `tool_find_courses_online` có mô tả *"Tìm kiếm khóa học trực tuyến liên quan tới kỹ năng còn thiếu. Sử dụng Tavily (search engine)"*.
- **Bước 2 - Định tuyến ngữ nghĩa:** Khi người dùng nhập liệu, GPT-4o sẽ tính toán xác suất ngữ nghĩa.
Ví dụ: Nếu câu hỏi liên quan đến sửa CV, vector ngữ nghĩa sẽ hướng luồng xử lý về tool `suggest_cv_improvements` thay vì `find_jobs`.
- **Bước 3 - Cơ chế dự phòng:** Trong trường hợp yêu cầu không khớp với bất kỳ tool nào (ví dụ: chào hỏi, câu hỏi xã giao, hoặc câu hỏi kiến thức chung không cần dữ liệu thời gian thực), Agent sẽ tự động chuyển về chế độ hội thoại thông thường để trả lời trực tiếp mà không tốn tài nguyên gọi Tool.

Đối với chế độ hội thoại thông thường, Agent sẽ dựa trên kiến thức nội tại của mô hình để trả lời cho người dùng.

c. Ưu điểm kỹ thuật so với Rule-based System

- **Tính linh hoạt:** Không cần viết hàng trăm dòng lệnh if-else để bắt từ khóa. Agent có thể hiểu các yêu cầu phức tạp như *"Tìm việc cho tôi, nhưng chỉ những việc nào không yêu cầu kinh nghiệm"*.
- **Khả năng mở rộng:** Khi cần thêm tính năng mới (ví dụ: Phòng vấn thử), chỉ cần viết thêm một hàm Python và bind vào Agent, không cần sửa đổi luồng logic chính.

=> Tóm lại, sau đây là mô hình cách hoạt động của 1 Agent khi nhận được yêu cầu từ người dùng:



2.2. Kỹ thuật Prompt Engineering & Output Handling:

Để đảm bảo tính ổn định và chính xác khi làm việc với LLM trong thực tế, hệ thống áp dụng chiến lược Instruction-based Prompting kết hợp với thiết kế Modular Tooling.

a. Thiết kế System Prompt và Định danh vai trò:

Hệ thống thiết lập ngữ cảnh cụ thể ngay từ khi khởi tạo Agent bằng tiếng Việt để đảm bảo sự nhất quán về tư duy ngôn ngữ cho Model.

- Gán vai trò (Role-playing): System Message định nghĩa rõ: "Bạn là AI Recruitment Expert chuyên nghiệp."
- Nhiệm vụ: Các nhiệm vụ được liệt kê rõ ràng trong ngữ cảnh: "Phân tích CV/JD, tính điểm, so sánh kỹ năng" và "Đề xuất chỉnh sửa CV".

=> Việc định danh này giúp Agent tập trung vào domain tuyển dụng, giới hạn phạm vi trả lời và tránh các phản hồi lan man không đúng trọng tâm.

b. Cơ chế ép kiểu dữ liệu đầu ra:

Hệ thống sử dụng kỹ thuật Direct Instruction ngay trong Prompt để kiểm soát định dạng.

- Prompt Constraint(sử dụng tiêu luật lệ): Trong các tool quan trọng như **tool_analyze_skills**, câu lệnh prompt chứa các luật lệ nghiêm ngặt: "CHỈ trả JSON, không thêm mô tả hoặc markdown", và định nghĩa rõ cấu trúc các field (cv_skills, jd_skills...).
- Manual Output Cleaning: Do bản chất LLM đôi khi vẫn trả về định dạng Markdown (ví dụ: ``json ... ``), Backend phải được trang bị logic xử lý hậu kỳ (Post-processing) để tự động cắt bỏ các ký tự thừa, trích xuất phần nội dung JSON hợp lệ trước khi đưa vào hàm **json.loads()**.

c. Kỹ thuật Chia nhỏ tác vụ:

Hệ thống chia nhỏ quy trình phân tích thành các Tool chuyên biệt:

- Bước 1 (Extraction): **tool_extract_text** chỉ làm nhiệm vụ đọc file PDF/Ảnh, không phân tích.
- Bước 2 Storage: **tool_store_cv** lưu trữ dữ liệu vào bộ nhớ đệm session.
- Bước 3 Scoring: **tool_calculate_similarity** sử dụng prompt riêng biệt chỉ để tính điểm số (trả về float).
- Bước 4 Deep Analysis: **tool_analyze_skills** sử dụng prompt riêng để chuyên sâu vào việc bóc tách kỹ năng.

=> Kết quả: Mỗi lần gọi LLM chỉ tập trung giải quyết một vấn đề nhỏ xác định, giúp độ chính xác tăng lên đáng kể và giảm thiểu ảo giác.

d. Cơ chế phòng vệ lỗi:

Hệ thống tích hợp khối lệnh quản lý lỗi (**try-except**) xung quanh mọi điểm tương tác với LLM.

- Trong trường hợp Model trả về JSON sai cú pháp, hệ thống sẽ không làm sập tiến trình mà sẽ trả về nội dung thô để người dùng vẫn đọc được thông tin, hoặc trả về giá trị mặc định an toàn.

=> Cơ chế này đảm bảo tính sẵn sàng cao cho ứng dụng Backend ngay cả khi Model hoạt động không ổn định.

2.3. Quản lý Trạng thái Hội thoại:

Một trong những thách thức lớn nhất khi triển khai Chatbot trên nền tảng web là bản chất **phi trạng thái** của giao thức HTTP. Mặc định, mỗi yêu cầu gửi đến FastAPI là độc lập, server không biết người dùng này là ai và trước đó họ đã nói gì.

=> Để biến Chatbot thành một "Mentor" có khả năng ghi nhớ và xâu chuỗi vấn đề, hệ thống đã triển khai cơ chế **Session-based Memory Management (quản lý trí nhớ phiên làm việc)** thông qua LangChain.

a. Kiến trúc Ánh xạ phiên làm việc:

Thay vì lưu trữ lịch sử chat dưới local (phía trình duyệt), hệ thống tập trung quản lý trạng thái tại Backend để đảm bảo tính bảo mật và đồng bộ.

- **Cơ chế định danh:** Mỗi khi người dùng truy cập trang web, hệ thống sẽ khởi tạo một **session_id** duy nhất (UID). ID này được gửi kèm trong Header hoặc Body của mọi API Request tiếp theo (ví dụ: /api/chat).

- **Lưu trữ trong bộ nhớ:** Tại Backend (agent_api.py), một cấu trúc dữ liệu từ điển toàn cục được sử dụng để lưu trữ lịch sử hội thoại của tất cả người dùng đang hoạt động
- **Truy vấn thông tin từ bộ nhớ:** Khi nhận request từ User A, hệ thống dùng session_id để "truy tìm" lại toàn bộ đoạn chat cũ trong store, nạp lại vào bộ nhớ đệm trước khi gọi GPT-4o.

b. Chiến lược Tiêm ngữ cảnh:

Sau khi lấy được lịch sử, hệ thống không gửi thô sơ cho AI mà sử dụng kỹ thuật **Prompt Chaining** để ghép nối ngữ cảnh một cách logic. Cấu trúc một Prompt hoàn chỉnh gửi tới GPT-4o bao gồm 3 lớp:

1. **Lớp System (Luật bất biến):** Chứa chỉ thị cốt lõi "*Bạn là chuyên gia tư vấn tuyển dụng...*". Lớp này luôn nằm đầu tiên để định hình nhân cách cho Agent.
2. **Lớp History (Ký ức ngắn hạn):** Chứa k cặp hội thoại gần nhất. Lớp này giúp AI hiểu các đại từ nhân xưng tham chiếu

Ví dụ: khi user hỏi "*Nó có nghĩa là gì?*", AI biết "*Nó*" ám chỉ nội dung vừa nói ở câu trước.

3. **Lớp Human (Truy vấn hiện tại):** Câu hỏi mới nhất của người dùng.

c. Quản lý Giới hạn Ngữ cảnh:

Mô hình GPT-4o có giới hạn về độ dài đầu vào. Nếu cuộc hội thoại kéo dài hàng giờ, việc gửi toàn bộ lịch sử sẽ gây ra 2 vấn đề:

- **Tràn bộ nhớ:** Gây lỗi API.
- **Chi phí cao & Độ trễ lớn:** Xử lý càng nhiều text càng tốn tiền và chậm.

Để giải quyết, hệ thống áp dụng cơ chế **Sliding Window Buffer** của LangChain:

- Hệ thống chỉ giữ lại N cặp hội thoại gần nhất (ví dụ: 10 câu gần nhất).
- Các hội thoại cũ hơn sẽ tự động bị loại bỏ (FIFO - First In First Out) khỏi bộ nhớ đệm gửi đi.

=> Điều này đảm bảo AI luôn nhớ được diễn biến gần đây nhất mà vẫn duy trì hiệu năng hệ thống ổn định.

d. Sự khác biệt giữa Chat thường và Chat theo Context:

Ví dụ:

Không có Memory:

- User: "Tôi biết code Python."
- User: "Gợi ý việc làm cho tôi đi."
- AI: "Bạn muốn tìm việc gì?" => (AI quên mất user biết Python).

Có Memory:

User: "Tôi biết code Python. Gợi ý việc làm cho tôi đi."

AI: "Dựa trên kỹ năng **Python** bạn vừa chia sẻ, đây là 3 việc làm Backend Developer phù hợp..." => (AI đã có database để tìm kiếm dựa trên thông tin mà ứng viên đã đưa.)

2.4. Tạo truy vấn tìm kiếm động:

Trong khi các mô hình LLM như GPT-4o có kiến thức rất rộng, điểm yếu chí mạng của chúng là **dữ liệu tĩnh (kiến thức không tự cập nhật trong lúc suy luận)**. GPT-4o chỉ biết những gì đã học trong quá khứ và hoàn toàn "mù" trước thông tin thời gian thực (ví dụ: tin tuyển dụng vừa đăng sáng nay). Để khắc phục, hệ thống áp dụng kỹ thuật **sinh nội dung có tăng cường truy xuất (Retrieval-Augmented Generation)** kết hợp với cơ chế sinh truy vấn động theo ngữ cảnh (**Dynamic Query Generation**) để lấy dữ liệu mới từ Internet.

a. Vấn đề của Tìm kiếm Từ khóa Truyền thống:

Người dùng thường có xu hướng nhập những câu lệnh rất tự nhiên và thiếu từ khóa trọng tâm.

- *Ví dụ Input:* "Tìm việc nào lương cao tí mà gần nhà em ở Quận 1 ấy."

Nếu search trực tiếp: Các công cụ tìm kiếm (Google/Tavily) sẽ bối rối với các từ như "tí", "em", "ấy", dẫn đến kết quả sai lệch hoặc không tìm thấy gì.

b. Cơ chế Dynamic Query Generation (sinh truy vấn động):

Hệ thống sử dụng một lớp Agent trung gian để "phiên dịch" ý định người dùng thành ngôn ngữ máy tìm kiếm. Quá trình này diễn ra qua 3 bước xử lý logic bên trong quy trình:

1. **Trích xuất & Tóm tắt:** Agent phân tích đoạn chat và hồ sơ CV của người dùng để rút trích các thực thể hoặc đặc trưng quan trọng:
 - *Kỹ năng chính:* Python, React (Trích từ CV).
 - *Địa điểm:* Quận 1, TP.HCM (Trích từ tin nhắn).
 - *Mong muốn:* Lương cao (High salary).
2. **Tái cấu trúc truy vấn:**

Từ các thực thể trên, LLM viết lại thành một chuỗi truy vấn tối ưu cho Search Engine.

- *Input*: "Tìm việc nào lương cao tí mà gần nhà em ở Quận 1 ấy."
- *Generated Query*: "Senior Python React Developer jobs in District 1 Ho Chi Minh City high salary"

3. Thực thi:

Chuỗi truy vấn mới này mới được gửi tới **Tavily Search API** để thực hiện tìm kiếm quét sâu (Deep Search).

c. Tích hợp RAG vào quy trình trả lời:

Sau khi Tavily trả về kết quả (thường là dạng JSON thô chứa tiêu đề, link, tóm tắt), hệ thống không hiển thị ngay mà tiếp tục xử lý qua RAG:

- **Truy xuất:** Lấy về Top 5-10 tin tuyển dụng phù hợp nhất từ API.
- **Tăng cường ngữ cảnh:**

Hệ thống "nhồi" toàn bộ dữ liệu thô này vào Prompt của GPT-4o dưới dạng một đoạn văn bản ngữ cảnh (Context Block).

Prompt: "Dưới đây là thông tin thị trường việc làm mới nhất tôi vừa tìm được: [Job 1...], [Job 2...]. Hãy dùng thông tin này để trả lời người dùng."

- **Sinh câu trả lời:**

GPT-4o đọc hiểu dữ liệu thô và tổng hợp lại thành một câu trả lời tự nhiên, có tư duy so sánh.

- *Output*: "Tôi tìm thấy 3 công việc phù hợp với bạn tại Quận 1. Vị trí A tại công ty X có mức lương hấp dẫn nhất, tuy nhiên yêu cầu thêm kỹ năng AWS..."

d. Cơ chế tự điều chỉnh hành vi:

Thay vì sử dụng các thuật toán "Hard-coded Retry" (if-else để thử lại), hệ thống tận dụng vòng lặp suy luận reasoning-acting của Agent:

- **Vòng lặp suy luận:** Khi Tool tìm kiếm trả về kết quả "Không tìm thấy" hoặc danh sách rỗng, Agent sẽ nhận thông báo này như một quan sát.
- **Cơ chế suy luận thích nghi theo ngữ cảnh:** Dựa trên System Prompt và ngữ cảnh, Model tự động suy luận nguyên nhân (ví dụ: "Từ khóa quá cụ thể").

- **Truy vấn lại động:** Ở vòng lặp tiếp theo (while loop), Agent sẽ tự quyết định gọi lại Tool tìm kiếm với một từ khóa thay thế tổng quát hơn mà không cần lập trình viên phải viết logic xử lý lỗi cụ thể.

=> Cách tiếp cận này giúp hệ thống linh hoạt hơn, có thể xử lý các tình huống tìm kiếm thất bại theo nhiều cách khác nhau tùy thuộc vào ngữ cảnh hội thoại cụ thể.

.

Chương 3: Phân tích và thiết kế hệ thống

3.1. Kiến trúc Hệ thống Tổng thể

a. Tầng phát triển ứng dụng:

Là nơi nhóm phát triển ứng dụng từ các ý tưởng ban đầu

- **Cursor và Antigravity:** là các IDE có tích hợp coding agent hiện đại, hỗ trợ rất nhiều trong việc thiết kế backend và frontend, cũng như debug các lỗi lặt vặt mặc dù nhiều lúc agent cũng tạo ra các bug hoặc thiết kế không phù hợp làm hệ thống bị rối
- **Github:** lưu trữ mã nguồn, hỗ trợ rollback version
- **Quy trình CI/CD:** sử dụng github action để xây dựng pipeline CI/CD gồm các bước sau:
 1. Cài đặt và kiểm lỗi nhanh sau khi member có thẩm quyền cho phép merge code vào main branch từ develop branch, nếu không có lỗi thực hiện chạy Dockerfile, build image backend và frontend với tag là staging, sau đó đưa lên dịch vụ AWS ECR để lưu trữ
 2. Member xác nhận triển khai lên staging server để kiểm thử các chức năng mới của ứng dụng, kích bản tự động tải images mới từ ECR về và triển khai bằng Docker Compose, sau đó xóa các images cũ trên staging server
 3. Nếu các chức năng mới đã hoạt động ổn thỏa trên staging server, member tạo version tag cho repository hiện tại, kích bản tạo ra 2 *images backend và frontend giống với bước 1 nhưng với tag là phiên bản chính thức do member đặt sau đó push lên ECR
 4. Member xác nhận deploy lên production server bằng cách chọn tag mong muốn. Kích bản tự động tải 2 images backend và frontend với tag tương ứng, triển khai lại ứng dụng với docker compose, loại

bỏ các images cũ trên production server. Member cũng có thể chọn rollback về version cũ nếu version hiện tại xảy ra lỗi

b. Tầng Ứng dụng & Logic

Là nơi ứng dụng xử lý yêu cầu của user theo thiết kế logic của lập trình viên

Frontend:

- **React:** Dùng để xây dựng phần giao diện người dùng (UI), Quản lý trạng thái hiển thị của ứng dụng và giao tiếp với backend thông qua API

Reverse Proxy:

- **Nginx:** cung cấp các file tĩnh Frontend cho trình duyệt của user. Nginx được cấu hình để chỉ chấp nhận yêu cầu truy cập bằng tên domain do cloudflare quản lý, ngăn chặn mọi yêu cầu truy cập bằng địa chỉ ip của backend server, tránh tình trạng bị tấn công DDOS và chuyển tiếp các yêu cầu API từ người dùng đến Backend server (FastAPI) đang chạy ẩn bên trong hệ thống thay để tránh các yêu cầu truy cập trực tiếp đến Backend server từ bên ngoài nhằm mục đích bảo mật hệ thống

Backend:

- **Python:** Là ngôn ngữ lập trình chính, được sử dụng để thiết kế luồng làm việc và các hàm xử lý logic của ứng dụng
- **FastAPI:** Định nghĩa các Endpoints để Frontend gọi xuống, đưa yêu cầu của user đến đúng với dịch vụ mà họ đang sử dụng. Trả kết quả xử lý từ Backend cho Frontend
- **Redis:** Dùng để quản lý session của mỗi user và lưu trữ context (CV, chat history, ...) của user trong khoảng thời gian là 1 giờ, nếu trong khoảng thời gian này user không tương tác với hệ thống nữa thì dữ liệu sẽ được xóa tự động. Backend cần đọc context của user từ Redis để biết "Người dùng này đang nói về CV nào?" mà không bắt người dùng phải upload lại.

c. Tầng Trí tuệ & Tích hợp Công cụ:

Đây là "bộ não" của hệ thống, không chỉ xử lý văn bản mà còn có khả năng "thị giác" máy tính, vượt trội hơn các hệ thống NLP truyền thống. kết nối "Trí tuệ nhân tạo" với các module xử lý chuyên biệt và dữ liệu thực tế.

1. GPT-4o Vision:

- Hệ thống sử dụng model gpt-4o với tham số temperature=0 để đảm bảo tính nhất quán và chính xác cao nhất cho các tác vụ phân tích.
- **Chức năng đa xử lý:** Khả năng xử lý đầu vào đa dạng. Không chỉ đọc text, hệ thống còn "nhìn" được cấu trúc thị giác của CV thông qua tool

tool_extract_text_from_file. Điều này giúp xử lý các CV dạng ảnh, infographic hoặc PDF phức tạp mà các thư viện OCR truyền thống (như Tesseract) thường thất bại.

2. Quản lý Ngữ cảnh Có trạng thái:

- Hệ thống duy trì trạng thái phiên làm việc thông qua Redis
- Mỗi Session ID được map với một key trong Redis chứa toàn bộ ngữ cảnh (CV, JD, Chat History). Điều này cho phép AI thực hiện các tác vụ chuỗi (Sequential Tasks): Người dùng chỉ cần upload CV một lần, Agent sẽ "nhớ" nội dung đó để thực hiện các tác vụ tiếp theo như: Tìm việc => Gợi ý công ty(2.3, quản lý trạng thái làm việc)

=> Chat tư vấn mà không cần dữ liệu đầu vào.

3. Lập luận Cấu trúc:

- Thay vì trả lời tự do, Agent bị ràng buộc bởi các **Prompt Template** phức tạp (như quy trình "6 BƯỚC ĐƠN GIẢN" trong hàm **analyze_cv_jd**).

=> Kỹ thuật này ép buộc mô hình tuân thủ quy trình nghiệp vụ (Business Logic) chặt chẽ: OCR + text preprocessing + pdf preprocessing => Lưu trữ => Tính điểm => Phân tích => đề xuất

Đường ống OCR(dành cho pdf):

- Hệ thống tích hợp PyMuPDF để trích xuất văn bản (Text Extraction) trực tiếp từ file PDF giúp tốc độ xử lý cực nhanh và chính xác với các định dạng text-based.

Đối với file dạng ảnh (PNG/JPG), hệ thống tự động chuyển sang sử dụng model GPT-4o Vision để "đọc" nội dung thông qua thị giác máy tính, đảm bảo xử lý được cả các CV dạng ảnh chụp hoặc thiết kế phức tạp.

Liên kết Hàm Module:

- Hệ thống sử dụng decorator `@tool` của LangChain để "đóng gói" các hàm Python thuần túy (tools_similarity, tools_skills) thành các giao diện mà AI có thể hiểu và thực thi.
- **Cơ chế và phòng vệ lỗi:** Mỗi tool đều được bọc trong khối try-except để đảm bảo khi một module con bị lỗi (ví dụ: lỗi tính toán vector), Agent vẫn hoạt động và báo cáo lỗi thay vì làm sập toàn bộ hệ thống backend.

d. Tầng triển khai và cơ sở hạ tầng của ứng dụng (Deployment & Infrastructure Layer):

- **AWS EC2:** là nơi nhóm thuê máy chủ từ AWS để vận hành ứng dụng, Loại Instance hiện tại đang sử dụng là m7i-flex.large (production server) và t3.small (staging server)
- **AWS ECR:** là dịch vụ cung cấp nơi để chứa các images của ứng dụng đã được đóng gói bằng Docker. Các dịch vụ sử dụng bao gồm EC2, ECR. Loại Instance được thuê trên AWS là m7i-flex.large (production server) và t3.small (staging server)
- **Docker:** công cụ quan trọng để đóng gói ứng dụng thành các images để đảm bảo sự nhất quán về môi trường chạy phù hợp cho ứng dụng
- **Cloudflare:** cung cấp tên miền cho public IP của server, đồng thời cung cấp dịch vụ truyền tin bằng giao thức HTTPS để mã hóa dữ liệu trên đường truyền giữa user và server nhằm bảo mật thông tin cho user

e. Tầng giám sát và kiểm thử:

- **Python:** là ngôn ngữ để viết script tự động ping đến server vào mỗi khoảng thời gian nhất định (VD: ping mỗi 1 phút), nếu server gặp lỗi (status code = 5xx hoặc timeout), script sẽ gửi thông báo về telegram
- **Crontab:** là một công cụ trên Linux để lập lịch chạy tự động cho script

3.2. Quy trình hoạt động của ứng dụng:

Trình bày tóm lược các bước hoạt động của ứng dụng từ khi user truy cập website, gửi CV đến khi user nhận được phản hồi của hệ thống. Quy trình này được chia thành 3 giai đoạn:

1. Khởi tạo kết nối từ browser của user đến website
2. Gửi CV, đoạn chat đến hệ thống và xử lý logic bên trong hệ thống
3. Trả phản hồi về user

+) **Giai đoạn 1: Khởi tạo kết nối từ browser của user đến website**

- User gửi yêu cầu truy cập vào tên miền website từ trình duyệt máy tính của họ, ở bước này trình duyệt của user sẽ tiến hành bắt tay TCP và bắt tay TLS với Cloudflare. Kể từ lúc này, mọi nội dung hay request do user gửi đi sẽ được mã hóa bằng chứng chỉ TLS và gửi đến Cloudflare
- Sau khi quá trình bắt tay hoàn tất, Cloudflare sẽ tiếp nhận request thực hiện các kiểm tra bảo mật (kiểm tra IP của user có trong blacklist hay không, nội dung của request có chứa mã độc hay không, ...). Nếu xác nhận request an toàn, thực hiện chuyển tiếp request đến địa chỉ public IP của server đăng sau tên miền

- Nginx ở server lắng nghe ở cổng 443 (HTTPS), đón request từ Cloudflare. Nhận thấy request là đường dẫn gốc '/', Nginx sẽ kết nối với container Frontend để lấy các file tĩnh html, css, js và trả về cho Cloudflare. Cloudflare sẽ tiếp tục trả về cho trình duyệt của user
- Các file tĩnh này sẽ chạy trên trình duyệt của người dùng, vẽ nên giao diện của website

+) Giai đoạn 2: Gửi CV, đoạn chat đến hệ thống và xử lý logic bên trong hệ thống:

- Khi user sử dụng các chức năng như gửi CV hoặc chat với AI, phần giao diện lúc này sẽ lấy dữ liệu của user gửi, đóng gói lại thành file JSON, gán endpoint của chức năng tương ứng (POST /api/analyze, POST /api/chat,...) và gửi đến server với con đường tương tự như giai đoạn 1 (-> Cloudflare -> Nginx -> Backend)
- Backend (cụ thể là FastAPI) nhận được file PDF/png hoặc text, thực hiện quá trình xử lý đầu vào và xử lý logic. Hệ thống không xử lý dữ liệu theo cách tuyến tính đơn giản mà áp dụng một quy trình **Đa bước (Multi-stage Pipeline)** kết hợp giữa xử lý ảnh kỹ thuật số và trí tuệ nhân tạo tạo sinh. Quy trình xử lý này được chia thành 3 bước chính: Tiếp nhận và trích xuất dữ liệu, Lưu trữ trạng thái phiên làm việc, và xử lý yêu cầu của user:

Bước 1: Tiếp nhận và trích xuất dữ liệu

Tiến hành đọc và trích xuất dữ liệu từ context do user gửi. Đây là bước tiền xử lý quan trọng nhất để đảm bảo AI nhìn thấy CV giống như mắt người nhìn, thay vì chỉ đọc các dòng text rời rạc. Logic này được thực thi trong hàm `tool_extract_text_from_file`.

Bước 2: Cơ chế lưu trữ trạng thái phiên làm việc

- Do kiến trúc Agent của LangChain có thể gọi nhiều tool liên tiếp, dữ liệu cần được duy trì xuyên suốt phiên làm việc.
- Trường hợp nếu user chưa có phiên làm việc: Backend tiến hành tạo ra một chuỗi ID ngẫu nhiên duy nhất, thực hiện lưu nội dung đã trích xuất vào Redis với key là ID vừa tạo. Sau đó trả về Cookie chứa ID trên cho user
- Trường hợp user đã có phiên làm việc: Bên user sẽ gửi nội dung kèm với Cookie nhận từ phần Backend trước đó. Backend dựa vào ID chứa trong Cookie để lấy các context mà user đã gửi trước đó được lưu trữ bởi Redis và gửi cho agent xử lý

Bước 3: Agent xử lý thông tin theo yêu cầu của user:

- Đây là nơi Agent nhận context của user và thực thi logic nghiệp vụ phức tạp thông qua **Prompt Engineering** trong các hàm như `analyze_cv_jd`, `find_jobs`, ...
- Phản hồi của agent từ phần xử lý logic cũng sẽ được lưu vào Redis với ID của user hiện tại

+) **Giai đoạn 3: Gửi trả phản hồi về user:**

- Sau khi nhận được phản hồi từ phần xử lý logic, Backend tiến hành chuyển đổi phản hồi sang dạng JSON và gửi trả về theo chiều ngược lại với khi gửi (Backend -> Nginx -> Cloudflare -> User)

3.3. Thiết kế các module chính và các tools tích hợp kèm theo

Hệ thống được cấu thành từ 5 module chức năng chính và các tools xử lý đi kèm, tương tác chặt chẽ với nhau thông qua cơ chế **Tool Calling** của LangChain và biến trạng thái toàn cục.

a. Module khởi tạo agent (`initialize_agent_api`)

- Module này sẽ khởi tạo agent để thực hiện các tác vụ xử lý logic.
- Model được sử dụng: GPT-4o
- Tích hợp thêm các tools được định nghĩa bên dưới

b. Module phân tích cv và jd (`analyze_cv_jd_api`) - TAB 1

- Đây là chức năng đầu tiên được sử dụng ngay lập tức khi user upload CV và JD. Thực hiện đánh giá, phân tích và so sánh giữa cv và jd bao gồm các bước thực hiện như sau:

Process Input & Store: Trích xuất và xử lý CV và JD do user gửi vào với các định dạng như PDF, png, text, ... Sử dụng `tool tool_extract_text_from_file`, `tool_process_text_input`, `tool_store_cv_text`, `tool_store_jd_text`

Calculate Similarity: Gọi tool `tool_calculate_match_score` để thực hiện đánh giá mức độ phù hợp bằng LLM Reasoning trên dữ liệu trong Redis Session.

Skill Gap Analysis: Gọi tool `tool_analyze_skills` để phân tích kỹ năng bằng GPT-4o. Tool này trả về phân tích chi tiết kỹ năng đạt được và còn thiếu.

External Knowledge Retrieval: Với danh sách missing_skills, Agent gọi `tool\_find\_courses\_online` để truy xuất dữ liệu thực tế từ Internet (Tavily Search), tìm kiếm các khóa học phù hợp từ Coursera/Udemy/edX kèm link truy cập trực tiếp.

Report Synthesis: Cuối cùng, Agent tổng hợp tất cả output từ các bước trên thành một báo cáo Markdown hoàn chỉnh theo format đã định nghĩa (#  KẾT QUẢ PHÂN TÍCH...).

c. Module tìm việc làm phù hợp(find_suitable_jobs_api) - TAB 2

- Đây là module thực hiện đánh giá khả năng được gọi đi phỏng vấn với vị trí ứng tuyển hiện tại, sau đó tìm thêm một vài công việc thích hợp khác cho user:
 - **Kiểm tra Điều kiện Tiên quyết:** Hàm kiểm tra ngay lập tức dữ liệu trong Redis Session. Nếu người dùng chưa phân tích CV (Tab 1), hệ thống chặn lại và báo lỗi ngay, tránh lãng phí token.
 - **Phân tích context:** Agent đọc lại nội dung CV từ Redis (đã lưu ở bước trước) để hiểu trình độ và chuyên môn của ứng viên.
 - **Đánh giá khả năng được gọi đi phỏng vấn với JD hiện tại:** Nêu rõ khả năng được gọi đi phỏng vấn, điểm mạnh của user so với công việc muốn ứng tuyển và những điều cần chuẩn bị thêm.
 - **Tìm kiếm các công việc khác đang tuyển dụng dựa trên CV:** Agent sử dụng `tool\_find\_jobs\_online`, sử dụng cơ chế Deep Search của Tavily để tìm kiếm các tin tuyển dụng đang tồn tại trên internet.

d. Module đánh giá và gợi ý sửa đổi CV (suggest_cv_improvements_api, analyze_cv_layout_api):

- Module này sẽ thực hiện phân tích và đánh giá bố cục và cách bày trí thông tin trong CV, gợi ý chỉnh sửa và bổ sung thêm một vài nội dung bị thiếu sót nếu cần thiết.

e. Module chat với AI (chat_with_agent_api):

- User có thể chat với AI để giải đáp các thắc mắc hiện tại của mình, cũng như luyện tập phỏng vấn với Chatbot này với các yêu cầu như ‘Dựa vào CV và JD mà tôi gửi, hãy soạn cho tôi 5-10 câu phỏng vấn thực tế và nhận xét câu trả lời của tôi’

f. Định nghĩa các tools được tích hợp cho agent:

Nhóm 1: Xử lý và Chuyển đổi Dữ liệu

Nhóm này chịu trách nhiệm làm sạch dữ liệu thô trước khi đưa vào bộ nhớ.

1. Tool: tool_extract_text_from_file (Trích xuất Đa phương thức)

- **Chức năng:** Đọc hiểu tài liệu từ đường dẫn file (Local Path). Xử lý linh hoạt cả PDF và Hình ảnh (PNG/JPG).
- **Logic kỹ thuật chuyên sâu:**
 - Input: Đường dẫn file (str).
 - Cơ chế PDF: Sử dụng PyMuPDF (fitz) để trích xuất text trực tiếp (`get_text()`) nhằm tối ưu tốc độ và độ chính xác với văn bản digital.
 - Cơ chế Vision: Chỉ áp dụng khi đầu vào là file ảnh (PNG/JPG). Gửi ảnh (Base64) tới gpt-4o với prompt đặc tả cấu trúc.
 - Output: Văn bản thô (Raw Text) đã được số hóa.

2. Tool: tool_process_text_input:

- **Chức năng:** Chuẩn hóa văn bản đầu vào dạng text (copy-paste).
- **Logic Kỹ thuật:**
 - Gọi module `tools_ocr.process_raw_text` để loại bỏ các ký tự lạ (artifacts), khoảng trắng thừa và chuẩn hóa Unicode (NFC) để tránh lỗi font tiếng Việt.

Nhóm 2: Lưu trữ và Quản lý Trạng thái

Nhóm này đóng vai trò bộ nhớ cho Agent, khắc phục nhược điểm quên ngay lập tức của mô hình LLM.

3. Tool: tool_store_cv_text (Ghim Dữ liệu CV)

- **Chức năng:** Lưu nội dung CV vào Redis Session của người dùng hiện tại.
- **Tầm quan trọng:** Giúp hệ thống tuân thủ nguyên tắc "**Write Once, Read Many**". Sau khi lưu, các tool khác có thể truy xuất dữ liệu này hàng chục lần mà không cần tốn chi phí OCR lại file gốc.

4. Tool: tool_store_jd_text (Ghim Dữ liệu JD)

- **Chức năng:** Tương tự như trên, nhưng dành cho Mô tả công việc (Job

Description). Lưu trữ nội dung JD vào Redis Session của người dùng hiện tại.

Nhóm 3: Phân tích, Tính toán và Đề xuất

Đây là nhóm suy luận, nơi diễn ra các thuật toán so khớp và tư duy.

5. Tool: `tool_calculate_match_score` (Thuật toán Tính điểm)

- **Chức năng:** Định lượng mức độ phù hợp giữa CV và JD.
- **Logic Kỹ thuật:**
 - *Truy xuất:* Tự động lấy dữ liệu từ Redis Session, ở nhóm 2.
 - *Cơ chế:* Sử dụng trực tiếp GPT-4o để đánh giá ngữ nghĩa và trả về điểm số từ 0.0 đến 1.0, thay thế cho các phương pháp tính khoảng cách Vector truyền thống.
 - *Output:* Một con số thực (float) từ 0.0 đến 1.0 (ví dụ 0.85 tương đương 85%)

6. Tool: `tool_analyze_skills` (Phân tích Khoảng cách Kỹ năng)

- **Chức năng:** So khớp từ khóa để tìm ra điểm mạnh và điểm yếu.
- **Logic Kỹ thuật:**
 - Sử dụng Prompt Engineering chuyên sâu để yêu cầu LLM bóc tách và so sánh danh sách kỹ năng, trả về JSON cấu trúc gồm `cv_skills`, `jd_skills`, `matched_skills`, và `missing_skills`.
 - *Output Format:* Trả về chuỗi cấu trúc đặc biệt `cv_skills: [...] ||| missing_skills: [...]`. Định dạng này giúp Agent dễ dàng parse (tách chuỗi) để đưa vào báo cáo cuối cùng.

7. Tool: `tool_suggest_jobs` (Đề xuất việc làm)

- **Chức năng:** Gợi ý danh sách vị trí công việc dựa trên phân tích năng lực.
- **Logic Kỹ thuật:**
 - Khác với tìm kiếm online, tool này sử dụng **Internal Knowledge** của GPT-4o.
 - Nó phân tích '`cv_text`' từ bộ nhớ để xác định Seniority (Junior/Senior) và Domain (Backend/Frontend), từ đó đề xuất các chức danh (Job Titles) chính xác nhất cho ứng viên.

8. Tool: Tavily Search Integration (Tích hợp Tìm kiếm Mở rộng)

- **Chức năng:** Kết nối Internet để lấy dữ liệu tuyển dụng thực tế thời gian thực.
- **Cơ chế:** Agent tạo ra các truy vấn tìm kiếm tối ưu hóa dựa trên CV, gửi tới Tavily API để quét qua hàng loạt nguồn dữ liệu và trả về danh sách

công việc đang tuyển dụng dưới dạng Markdown kèm link chi tiết.

Chương 4: THỰC NGHIỆM VÀ ĐÁNH GIÁ

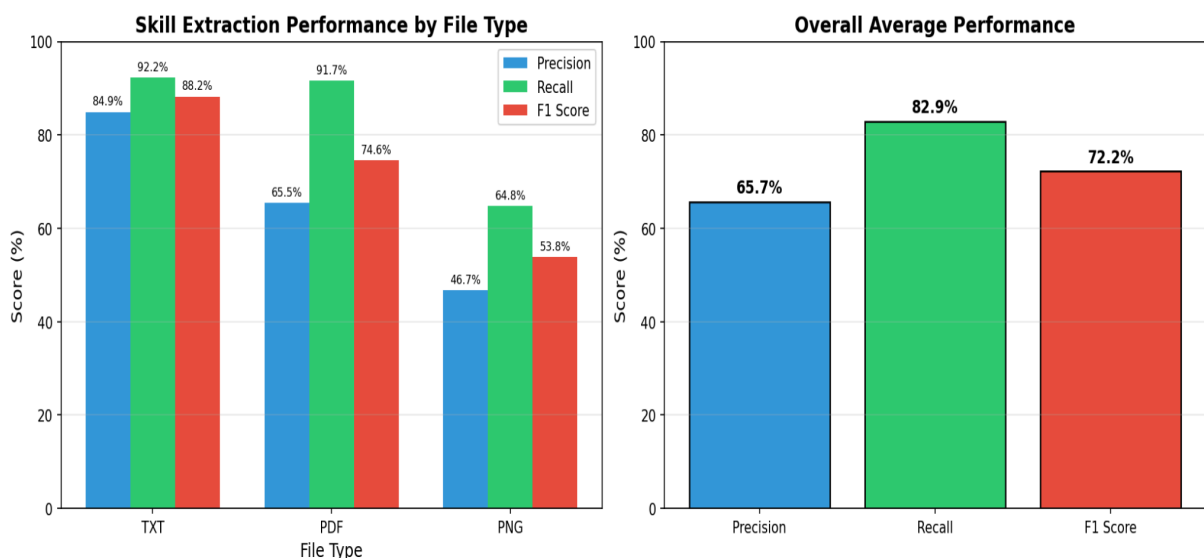
- Đánh giá các chức năng như lấy kỹ năng, tìm kiếm khóa học và tìm kiếm công việc phù hợp trên local host.
- Đánh giá hiệu năng hệ thống
- Dataset: 10 file ảnh, 9 file pdf, 10 file text

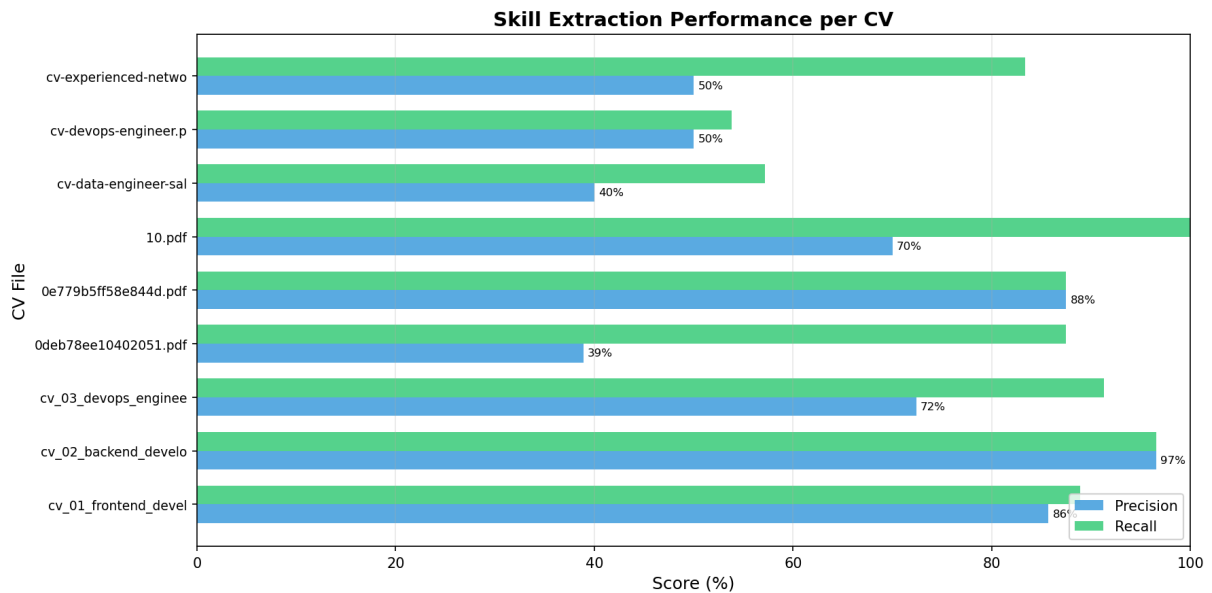
Nguồn dataset:

- Kaggle:
- Từ bạn bè:
- enhancv.com
- resumeworded.com
- topcv.vn
- beamjobs.com
- Agent sinh ra các file txt cho các mô tả công việc và các cv mẫu

4.1 Đánh giá các chức năng của Agent:

1. Tool: tool_extract_text_from_file (Trích xuất Đa phương thức):





Hệ thống sử dụng phương pháp **Đánh giá dựa trên Ground Truth**. Đây là tiêu chuẩn phổ biến trong các bài toán NLP/ML (như NER - Named Entity Recognition) để đo lường hiệu suất của hệ thống trích xuất thông tin so với dữ liệu nhãn chuẩn.

Để đánh giá chất lượng mô hình, 3 chỉ số chính được sử dụng:

1. Precision(Độ chính xác):

$$Precision = \frac{\text{Số skill đúng (True Positives)}}{\text{Tổng số skill AI trích xuất (True Positives + False Positives)}}$$

Ý nghĩa: Trả lời câu hỏi "Khi AI nói một skill, skill đó có đúng không?".

Đặc điểm:

- Precision cao => AI ít đưa ra kết quả ảo.
- Precision thấp => AI hay đưa ra các kỹ năng không có trong CV.

2. Recall(Độ bao phủ):

Đo lường mức độ **đầy đủ** của thông tin trích xuất.

$$Recall = \frac{\text{Số skill đúng (True Positives)}}{\text{Tổng số skill trong Ground Truth (True Positives + False Negatives)}}$$

Ý nghĩa: Trả lời câu hỏi "AI có tìm được hết các skill trong CV không?".

Đặc điểm:

- Recall cao $\Rightarrow$ AI ít bỏ sót thông tin quan trọng.
- Recall thấp $\Rightarrow$ AI hay bị miss skill.

3. F1 Score (Trung bình hài hòa)

Là chỉ số cân bằng giữa Precision và Recall.

$$F1 = 2 \times \frac{Precision \times Recall}{Precision + Recall}$$

- **Ý nghĩa:** Đánh giá hiệu suất tổng thể. Điểm số lý tưởng là khi AI vừa chính xác vừa đầy đủ.

+) Cách tính toán:

Hệ thống phân loại các trường hợp trích xuất như sau:

- **TP (True Positive):** AI trích xuất đúng skill có trong Ground Truth.
- **FP (False Positive):** AI sinh ra skill không có trong Ground Truth.
- **FN (False Negative):** AI bỏ sót skill có trong Ground Truth.

Ví dụ minh họa:

Có CV_skills như sau:

Python, React, Docker

- **Ground Truth (Thực tế):** [Python, React, Docker, AWS]
- **AI Predicted (Dự đoán):** [Python, React, Docker, Kubernetes]

Phân tích:

- **Đúng (TP = 3):** Python, React, Docker.
- **Bỏ sót (FN = 1):** AWS (Có trong thực tế nhưng AI không thấy).
- **Bịa (FP = 1):** Kubernetes (AI tự thêm vào, không có trong thực tế).

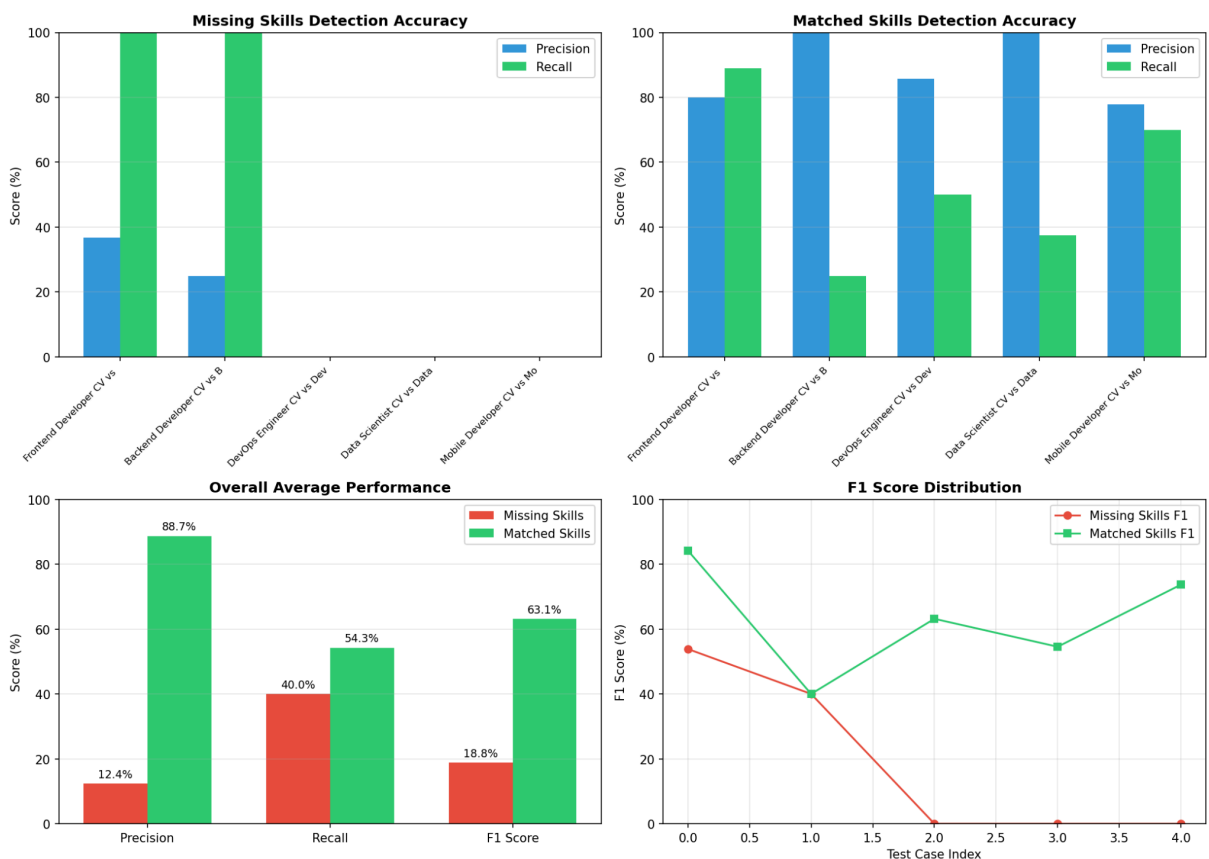
Kết quả:

- Precision = $3/4 = 75\%$
- Recall = $3/4 = 75\%$
- F1 Score = 75%

$\Rightarrow$ Kết quả thực nghiệm:

Format	Precision	Recall	F1 Score	Đánh giá
TXT	84.89%	92.25%	88.20%	Xuất sắc
PDF	65.46%	91.67%	74.57%	Recall tốt, nhưng Precision cần cải thiện
PNG	46.67%	64.77%	53.80%	Cần cải thiện cả hai

2. Tool: tool_analyze_skills (Phân tích Khoảng cách Kỹ năng/CV):



Hệ thống đánh giá dựa trên tập dữ liệu Ground Truth (các cặp CV-JD đã được con người gán nhãn). Bài toán được mô hình hóa dưới dạng tập hợp:

Gọi $S(JD)$ là tập hợp các kỹ năng yêu cầu trong JD. Gọi $S(CV)$ là tập hợp các kỹ

Chúng ta đánh giá hai mục tiêu riêng biệt:

1. **Matched Skills (Kỹ năng khớp):** Là giao của hai tập hợp:

$$S_{Matched} = S_{JD} \cap S_{CV}$$

2. **Missing Skills (Kỹ năng thiếu):** Là hiệu của tập JD và tập CV.

$$S_{Missing} = S_{JD} \setminus S_{CV}$$

Hệ thống sử dụng bộ 3 chỉ số tiêu chuẩn cho bài toán Information Retrieval:

A. Precision (Độ chính xác)

Đo lường độ tin cậy của những gì AI dự đoán.

$$Precision = \frac{TP}{TP + FP}$$

- **Đối với Matched Skills:** Trong số các skill AI báo là "Khớp", bao nhiêu % là đúng?
- **Đối với Missing Skills:** Trong số các skill AI báo là "Thiếu", bao nhiêu % thực sự thiếu?
- Precision thấp => AI báo nhầm (skill ứng viên có mà AI lại báo thiếu).

B. Recall (Độ bao phủ)

Đo lường khả năng tìm kiếm thông tin của AI.

$$Recall = \frac{TP}{TP + FN}$$

- **Đối với Matched Skills:** AI tìm ra được bao nhiêu % số skill khớp thực tế?

- **Đối với Missing Skills:** AI phát hiện được bao nhiêu % số skill còn thiếu?
 - Recall thấp => AI bỏ sót nhiều skill quan trọng mà JD yêu cầu.

C. F1 Score (Trung bình hài hòa)

Chỉ số cân bằng tổng thể.

$$F1 = 2 \times \frac{Precision \times Recall}{Precision + Recall}$$

+) Ma trận nhầm lẫn:

	Thực tế thiếu (Expected)	Thực tế KHÔNG thiếu
AI báo Thiếu	TP (True Positive) (AI đúng)	FP (False Positive) (AI sai - Báo nhầm)
AI im lặng	FN (False Negative) (AI sai - Bỏ sót)	TN (True Negative) (AI đúng)

Ví dụ minh họa:

- Expected (Thực tế): [Angular, Ember, iOS]

- AI Predicted (Dự đoán): [Angular, Ember, Mobile Dev]
- Phân tích:
 - TP = 2 (Angular, Ember)
 - FP = 1 (Mobile Dev - AI sử dụng sai từ)
 - FN = 1 (iOS - AI bỏ sót)

Dữ liệu thử nghiệm bao gồm 15 cặp CV-JD (Test Cases) được chia thành:

1. **Matching Pairs (1-10):** CV và JD cùng lĩnh vực (VD: Frontend CV vs Frontend JD).
 2. **Mismatch Pairs (11-15):** CV và JD khác lĩnh vực để kiểm tra khả năng xử lý biên.
1. **Load Test Cases:** Đọc file cấu hình test_cv_jd_analyze.json.
 2. **Execute:** Chạy hàm analyze_cv_jd(cv_text, jd_text) cho từng cặp.
 3. **Compare:** So sánh kết quả đầu ra của AI với expected_missing_skills và expected_matched_skills.
 4. **Calculate Stats:** Tính toán Precision, Recall, F1 trung bình trên toàn bộ dataset.

=> Kết quả và hạn chế:

Dựa trên kết quả chạy sơ bộ, chúng ta nhận thấy một số vấn đề quan trọng cần cải thiện:

Metric	Missing Skills	Matched Skills	Nhận xét
Precision	~12.37% (Thấp)	~88.70% (Cao)	AI hay báo nhầm skill thiếu.
Recall	~40.00% (TB)	~54.28% (Khá)	AI bỏ sót khá nhiều skill.
F1 Score	18.77%	63.12%	Hiệu suất Matched tốt hơn Missing.

Hạn chế cốt lõi:

1. Vấn đề ngữ nghĩa:

- Phương pháp so sánh chuỗi hiện tại không hiểu được từ đồng nghĩa.
- Ví dụ: JD yêu cầu "React Native", AI tìm ra "Mobile Development" => Hệ thống tính là sai (False Positive) dù về nghĩa có thể chấp nhận được.

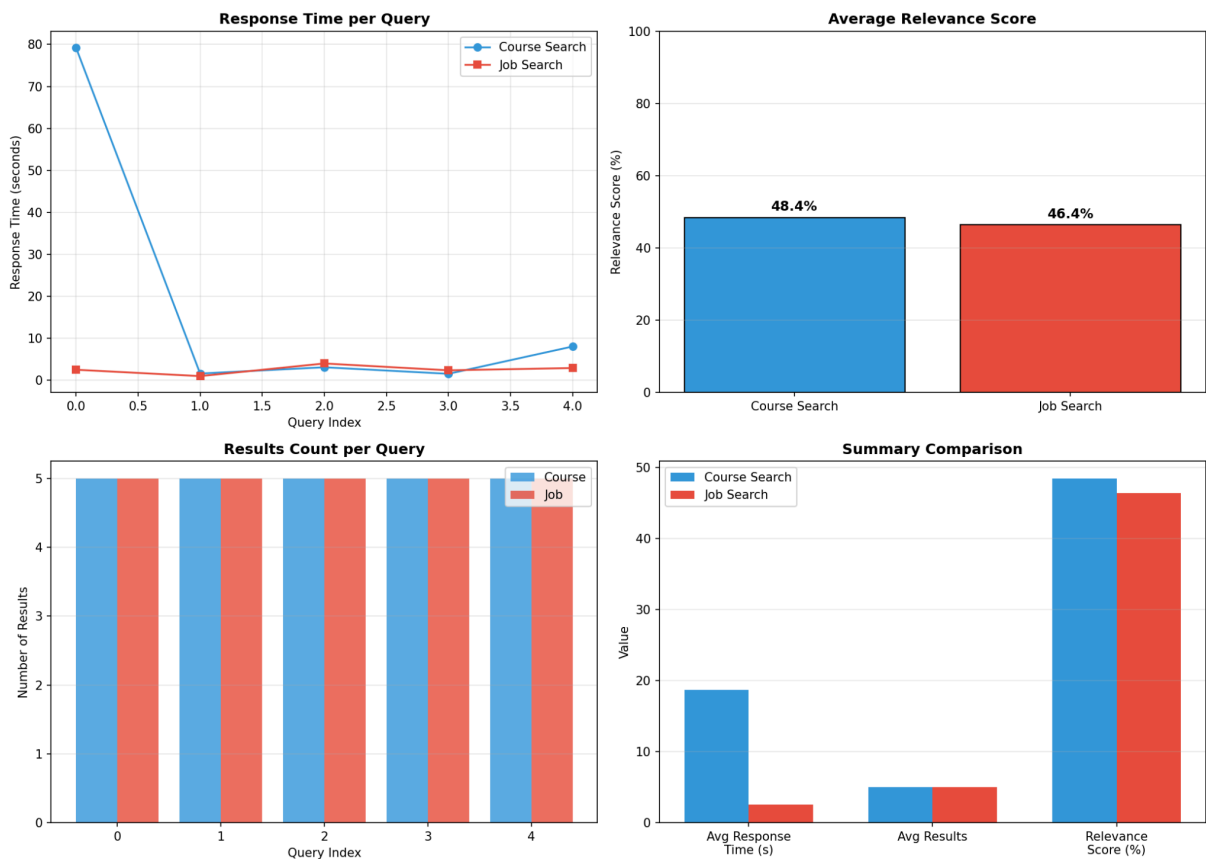
2. Cách diễn đạt khác nhau:

- "iOS" vs "iOS Development" vs "iPhone App Dev".

3. Chất lượng Ground Truth:

- Việc gán nhãn thủ công có thể mang tính chủ quan.

3. Tool: tool_suggest_jobs và suggest_courses(Đề xuất khóa học và việc làm)



Hệ thống sử dụng Tavily Search API làm engine tìm kiếm nền tảng. Quy trình đánh giá dựa trên việc thực thi tập dữ liệu mẫu và đo lường các chỉ số hiệu suất kỹ thuật cũng như độ liên quan của nội dung.

Số lượng mẫu thử nghiệm: 40 queries (20 Course + 20 Job).

Giới hạn kết quả: Tối đa 5 kết quả/query.

+) Các chỉ số đánh giá:

Để đảm bảo tính khách quan, hệ thống đo lường 4 chỉ số chính:

A. Success Rate (Tỷ lệ thành công)

Đo lường độ ổn định của hệ thống và API.

$$SuccessRate = \frac{\text{Số query thành công}}{\text{Tổng số query}} \times 100\%$$

- **Mục tiêu:** > 95%.
- **Ý nghĩa:** Nếu < 100%, cần kiểm tra lại kết nối mạng, API Key hoặc Rate Limit.

B. Response Time (Thời gian phản hồi)

Đo lường độ trễ (latency) từ lúc gửi yêu cầu đến khi nhận kết quả.

Các chỉ số con: Avg, Min, Max.

Thang đánh giá:

- < 2s: Xuất sắc.
- 2s - 5s: Chấp nhận được.
- > 5s: Cần tối ưu hóa (Cache, Async).

C. Relevance Score (Độ liên quan)

Đánh giá chất lượng nội dung trả về dựa trên phương pháp Keyword Matching (Khớp từ khóa).

Quy trình tính toán:

1. Tách query gốc thành tập hợp các từ khóa (Tokens).
2. Kiểm tra sự xuất hiện của từ khóa trong Title và Snippet của kết quả.
3. Tính điểm theo công thức:

$$Relevance = \frac{\sum(\text{Số từ khóa tìm thấy trong mỗi kết quả})}{\text{Tổng số từ khóa} \times \text{Tổng số kết quả}}$$

Ví dụ minh họa:

- Query: *"Python programming"* (2 từ khóa).
- Kết quả 1: "Learn Python" (Khớp 1/2 => 50%).
- Kết quả 2: "Python Programming Course" (Khớp 2/2 => 100%).
- => Relevance trung bình: $(50\% + 100\%) / 2 = 75\%$.
- Mục tiêu: > 50% (Do hạn chế của phương pháp keyword matching, mức 50-70% được coi là khá tốt).

D. Number of Results (Số lượng kết quả)

Đảm bảo API trả về đủ dữ liệu cho người dùng lựa chọn.

- Mục tiêu: 4 - 5 kết quả/query.

+) Dữ liệu thử nghiệm:

Dữ liệu đầu vào được lưu trong test_search_queries.json với cấu trúc đa dạng để đảm bảo tính bao quát:

Loại Tìm Kiếm	Các tiêu chí phân loại (Criteria)	Ví dụ
Course Search	Category (AI, DevOps...), Difficulty, Platform	{"query": "Python for beginners", "level": "Easy"}
Job Search	Seniority (Junior, Senior...), Location, Stack	{"query": "Remote Backend Java Jobs", "loc": "Remote"}

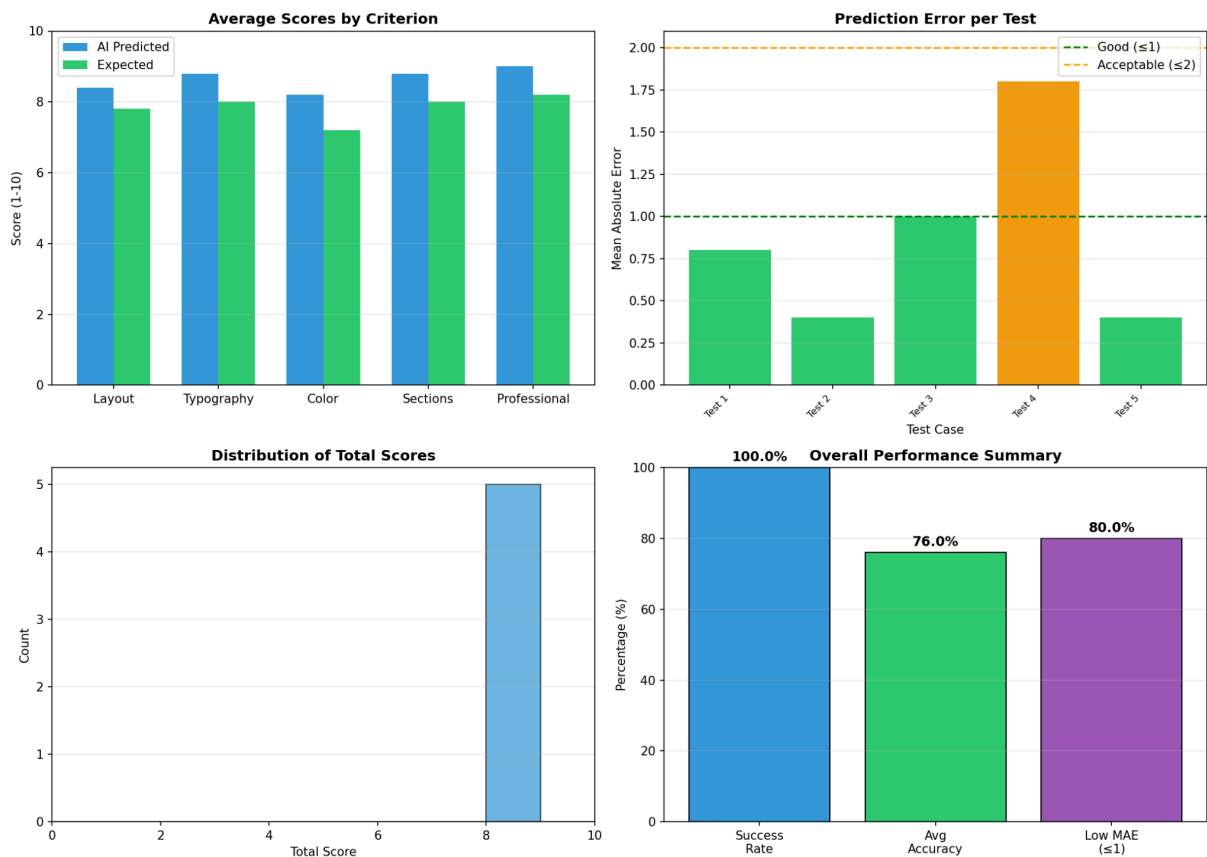
+) Kết quả đầu ra:

Sau khi chạy benchmark, hệ thống xuất ra 2 file báo cáo:

1. benchmark_search_results.json(github):
 - Chứa log chi tiết từng query (Status, Time, Relevance).
 - Thống kê tổng hợp (Aggregate metrics) cho cả 2 loại tìm kiếm.
2. benchmark_search_chart.png(ảnh trên):

- Biểu đồ trực quan hóa so sánh thời gian phản hồi (Response Time).
- Biểu đồ điểm độ liên quan (Relevance Score).

4. Tool suggest_cv_improvement(gợi ý layout):



Khác với việc so khớp từ khóa, đánh giá Layout đòi hỏi khả năng xử lý thị giác máy tính để hiểu về cấu trúc không gian.

- Công nghệ: GPT-4o Vision (Multimodal AI).
- Dữ liệu đầu vào: 20 Test Cases (bao gồm cả định dạng ảnh PNG và tài liệu PDF).
- Mục tiêu: So sánh điểm số AI chấm với điểm số kỳ vọng (Ground Truth - do con người thiết lập) trên thang điểm 1-10.

Nhất quyết 5 tính toán diện để đánh giá 1 CV layout.

Tiêu chí	Các yếu tố thành phần	Thang điểm (1-10)

1. Layout Structure	Phân chia cột (1 vs 2 cột), Sidebar, khoảng trắng, phân cấp thị giác.	1-3: Lộn xộn. 8-10: Rõ ràng, dẫn dắt mắt người đọc tốt.
2. Typography	Lựa chọn Font, kích thước tiêu đề/nội dung, khả năng đọc.	1-3: Khó đọc, font lỗi. 8-10: Chuyên nghiệp, phân cấp rõ.
3. Color Scheme	Sự hài hòa, độ tương phản, tính chuyên nghiệp của bảng màu.	1-3: Màu lòe loẹt/khó nhìn. 8-10: Màu nhân tinh tế, hài hòa.
4. Section Org.	Luồng thông tin, tiêu đề mục rõ ràng, sắp xếp hợp lý.	1-3: Sắp xếp tùy tiện. 8-10: Logic, dễ tìm thông tin.
5. Professionalism	Ấn tượng tổng thể, sự phù hợp với ngành nghề, tính hiện đại.	1-3: Nghiệp dư. 8-10: Executive level, ấn tượng mạnh.

Vì đánh giá thẩm mỹ mang tính chủ quan, chúng ta không dùng "Đúng/Sai" tuyệt đối mà sử dụng "Độ lệch" (Error Margin).

A. MAE (Mean Absolute Error - Sai số tuyệt đối trung bình):

Đo lường khoảng cách trung bình giữa điểm AI chấm và điểm con người chấm.

$$MAE = \frac{\sum_{i=1}^N |Score_{AI} - Score_{Human}|}{N}$$

N: Tổng số tiêu chí (5 tiêu chí).

Đánh giá:

- MAE ≤ 1.0 : Xuất sắc (AI chấm gần như người).
- MAE > 2.0 : Cần cải thiện (AI đang đánh giá sai lệch nhiều).

B. Accuracy (Độ chính xác trong ngưỡng dung sai)

Tỷ lệ phần trăm các tiêu chí mà AI chấm lệch không quá ± 1 điểm so với đáp án.

$$Accuracy_{\pm 1} = \frac{\text{Số tiêu chí có } |Error| \leq 1}{\text{Tổng số tiêu chí}} \times 100\%$$

- Ý nghĩa: Chấp nhận sự chênh lệch nhỏ (ví dụ: Người chấm 8, AI chấm 9 -> Vẫn tính là Đạt).

C. Total Score (Điểm tổng hợp)

Điểm trung bình của CV dùng để xếp hạng.

$$TotalScore = \frac{Layout + Typo + Color + Section + Prof}{5}$$

Benchmark Progress

1. Input: Load file cấu hình test_cv_layout_analyze.json chứa đường dẫn file CV và điểm số kỳ vọng.
2. Processing:

- Chuyển đổi PDF => Ảnh (nếu cần).
 - Gửi ảnh tới GPT-4o Vision kèm Prompt chuyên gia thiết kế.
3. Analysis: Nhận về JSON chứa điểm số và nhận xét.
 4. Comparison: Tính toán MAE và Accuracy cho từng Test Case.

=> Kết quả mẫu và phân tích:

Ví dụ:

SUMMARY REPORT

=====
Total Tests: 5 | Successful: 5

SCORE PREDICTION ACCURACY:

- Avg MAE: 0.88 (Rất tốt, sai số chưa đến 1 điểm)
- Avg Accuracy (± 1): 76.00% (Khá tin cậy)

AVG PREDICTED SCORES:

- Layout: 8.4 • Typography: 8.8
 - Color: 8.2 • Professionalism: 9.0
- =====

Nhận xét:

- Chỉ số **MAE** = **0.88** cho thấy GPT-4o Vision có khả năng cảm thụ thẩm mỹ rất sát với tiêu chuẩn con người đặt ra trong bộ Test.
- Điểm **Professionalism** trung bình 9.0 cho thấy AI có xu hướng đánh giá tay không khắc khe hoặc bộ dữ liệu test đang thiên về các CV chất lượng cao.

4.2. Đánh giá hiệu năng hệ thống:

4.2.1 Đánh giá phần trăm mức sử dụng CPU với số lượng user tương ứng:

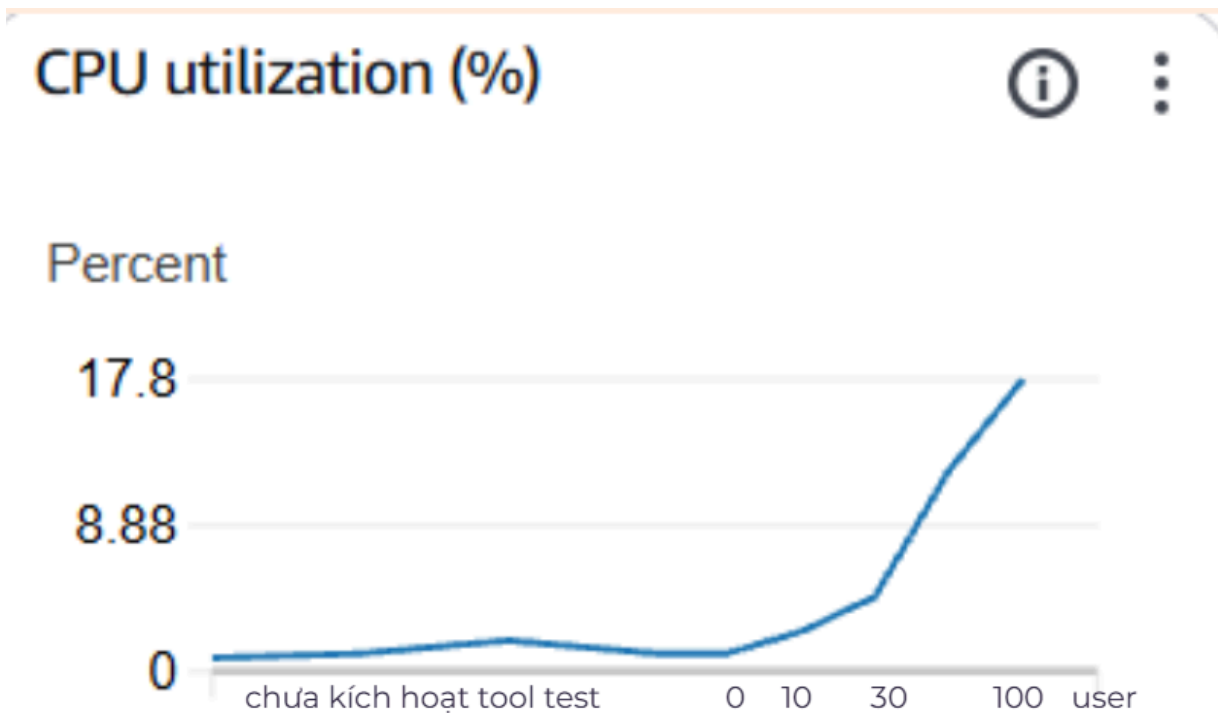
Công cụ sử dụng

- Locust là một công cụ kiểm thử chịu tải mã nguồn mở được viết bằng ngôn ngữ Python. Nguyên lý hoạt động của nó là sinh ra hàng nghìn, hàng triệu người dùng ảo cùng lúc để tấn công vào website/API, xem server chịu đựng được đến đâu trước khi sập.
- AWS CloudWatch là một dịch vụ giám sát và quan sát của AWS. Nhiệm vụ của công cụ này tại phần đánh giá này là đo CPU Utilization (Server đang dùng bao nhiêu % bộ vi xử lý)

Hạn chế: Do để tiết kiệm ngân sách mà nhóm đầu tư vào các dịch vụ của AWS và API của OPENAI, trong phần đánh giá này, chỉ test với số lượng user ảo lần lượt là 10, 30, 50 và 100 (sử dụng cùng lúc)

Kết quả đánh giá: Với mức kiểm tra khoảng 100 user truy cập cùng lúc và loại Instance đang sử dụng là m7i-flex.large (2 vCPU, 8 GiB RAM), kết quả cho thấy CPU Utilization = 17.8%. Điều này có nghĩa là trong 1 giây, server chiếm dụng CPU khoảng 0.178 giây để xử lý các yêu cầu từ user và khoảng thời gian còn lại thì CPU ở trạng thái chờ

Kết luận: Hệ thống có thể chịu tải thêm khoảng 4-5 lần hiện tại (khoảng 500 user sử dụng cùng lúc). Con số này là dư thừa đối với phạm vi một đồ án môn học vì nhóm vẫn chưa có nhiều kiến thức và kinh nghiệm về việc lựa chọn dịch vụ phần cứng hợp lý và một phần cũng vì nhóm muốn đem lại trải nghiệm tốt nhất cho mọi người tại buổi demo. Trong tương lai, nhóm sẽ cân nhắc lựa chọn Instance phù hợp hơn với số lượng user hiện tại và có thể áp dụng thêm chức năng scaling số lượng server nếu cần thiết.

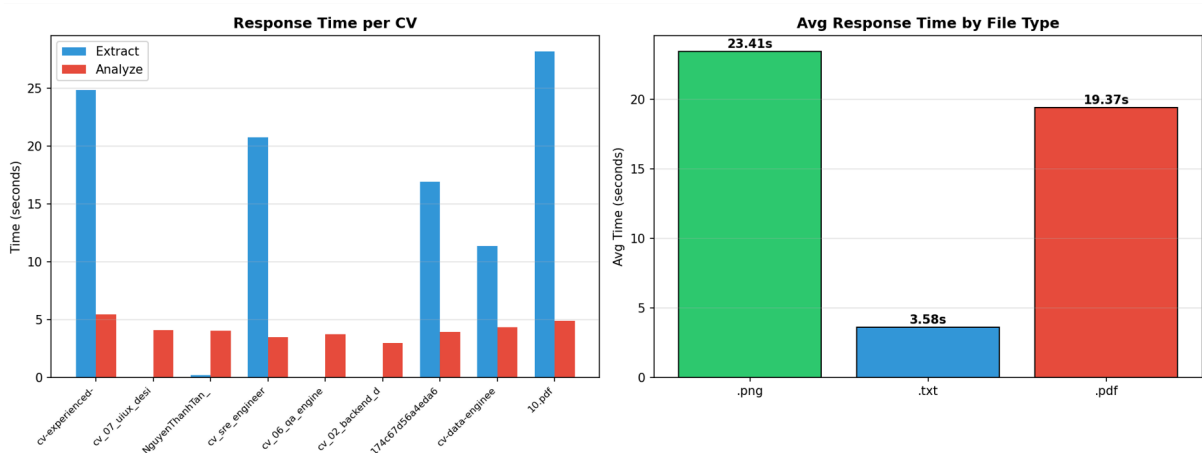


4.2.2 Đánh giá thời gian xử lý trung bình cho một yêu cầu phân tích CV

Cách thức thực hiện: Sử dụng ngẫu nhiên 10 mẫu CV có sẵn và đo đạc thời gian từ lúc CV được gửi vào cho đến lúc phản hồi được trả về hoàn tất

Kết quả: Trung bình thời gian xử lý của các file có định dạng png là cao nhất (Do phải gọi đến API của GPT-4o-Vision để trích xuất nội dung trước), sau đó là đến các file PDF (trích xuất bằng thư viện PyMuPDF) và thấp nhất là các file .txt (chỉ cần lấy trực tiếp nội dung trong file và gửi đi)

Kết luận: Nhóm có dự định rút ngắn thời gian xử lý các yêu cầu bằng cách nghiên cứu, tìm hiểu các open-source model có khả năng suy luận và so sánh mạnh mẽ để có thể tự host model ngôn ngữ thay vì phải call API của dịch vụ bên ngoài (giảm thiểu được độ trễ khi yêu cầu của user di chuyển giữa server ứng dụng và dịch vụ như OPENAI). Cũng như thay thế việc xử lý ảnh PNG bằng GPT-4o-Vision bằng các model thị giác (open-source) nhẹ hơn và nhanh hơn



Chương 5: KẾT LUẬN VÀ HƯỚNG PHÁT TRIỂN

5.1 Kết luận (Conclusion)

Sau quá trình nghiên cứu và thực hiện đồ án "AI Resume Analyzer v3.0", nhóm đã đạt được những kết quả khả quan, hoàn thành các mục tiêu đề ra ban đầu về việc xây dựng một hệ thống hỗ trợ tuyển dụng thông minh.

- **Kết quả đạt được:**

- Xây dựng thành công hệ thống phân tích CV tự động dựa trên kiến trúc **Agentic AI**, kết hợp sức mạnh của **LangChain** để điều phối tác vụ.
- Giải quyết triệt để bài toán xử lý đa định dạng tài liệu (PDF, Image) nhờ quy trình OCR lai tích hợp **GPT-4o Vision**.
- Tạo ra một trải nghiệm người dùng liền mạch với giao diện React hiện đại và Backend FastAPI hiệu năng cao.

5.2. Hạn chế và Thách thức (Limitations & Challenges)

Trong quá trình phát triển, nhóm nghiên cứu đã đối mặt với những thách thức lớn về việc lựa chọn công nghệ lõi, dẫn đến một số thay đổi chiến lược quan trọng:

- **Thách thức về Hạ tầng & Mô hình (Infrastructure & Model Selection):**
 - Ban đầu, nhóm dự định triển khai các mô hình ngôn ngữ mã nguồn mở (**Open-source LLMs**) từ cộng đồng Hugging Face như *Llama-2-7b*, *Mistral-7B* hoặc *Gemma*. Mục tiêu là để tự chủ hoàn toàn về mô hình AI.
 - Tuy nhiên, rào cản lớn nhất là **chi phí tính toán cost**. Để có thể chứa và vận hành mượt mà các model này với tốc độ phản hồi chấp nhận được, hệ thống yêu cầu hạ tầng phần cứng rất mạnh (High-end GPUs với VRAM lớn), buộc phải thuê các GPU Server đắt đỏ (như AWS EC2 g4dn/p3 instances). Điều này vượt quá ngân sách và quy mô cho đồ án môn học này, nhóm cũng đã thử nghiệm các model với kích thước nhỏ hơn nhưng đánh đổi về độ chính xác là quá lớn, không thể chấp nhận.
 - Rào cản thứ hai là nguồn data thích hợp để huấn luyện các model trên là không nhiều. Có dataset lên đến 1 triệu mẫu nhưng định dạng để huấn luyện cho các chức năng mà nhóm đề ra là không phù hợp. Có dataset phù hợp về định dạng nhưng số mẫu quá ít, không đủ để đạt độ chính xác chấp nhận được
- **Giải pháp thay thế (Strategic Pivot):**
 - Nhóm quyết định chuyển hướng sang sử dụng phương pháp tích hợp API của các mô hình thương mại (**Proprietary Model API**), cụ thể là **OpenAI GPT-4o**.
 - Việc kết hợp với **LangChain** để xây dựng các **AI Agents** cho phép hệ thống tận dụng khả năng suy luận top đầu thế giới hiện nay mà không cần lo lắng về hạ tầng phần cứng (Serverless approach). Dù đánh đổi bằng chi phí gọi API (Token cost), nhưng đây là giải pháp phù hợp về mặt hiệu năng/chi phí (Cost-Performance) trong giai đoạn phát triển hiện tại và cũng như để làm quen trong việc sử dụng api của các ông lớn trong lĩnh vực AI để tạo ra sản phẩm thực tế.
- **Hạn chế tồn đọng:**
 - **Phụ thuộc bên thứ 3:** Hệ thống hoạt động dựa hoàn toàn vào sự ổn định của OpenAI API và Tavily API. Nếu các dịch vụ này bảo trì hoặc thay đổi chính sách giá, hệ thống sẽ bị ảnh hưởng trực tiếp.
 - **Độ trễ (Latency):** Do phải gọi API qua mạng internet và chờ phản hồi từ Server nước ngoài, thời gian xử lý một CV đôi khi có thể kéo dài (5-10 giây), chưa đạt mức thời gian thực lý tưởng.

5.3. Hướng phát triển (Future Work)

Để khắc phục các hạn chế trên và nâng cao giá trị sản phẩm, nhóm đề xuất các hướng phát triển trong tương lai:

- **Phỏng vấn ảo (AI Mock Interview):** Tích hợp công nghệ **Voice-to-Text (STT)** và **Text-to-Speech (TTS)** để AI có thể phỏng vấn thử ứng viên bằng giọng nói, giúp luyện tập kỹ năng trả lời phỏng vấn.
- **Thiết kế lộ trình học tập và thực hành cho ứng viên:** dựa trên các kỹ năng đã có và cong thiếu sót hiện tại cũng như vị trí việc làm mong muốn, thiết kế nên một lộ trình chi tiết, chính xác và bền vững cho ứng viên
- **Gợi ý các sự kiện, kì thi và chứng chỉ ý nghĩa:** dựa trên các kỹ năng đã có và cong thiếu sót hiện tại cũng như vị trí việc làm mong muốn, gợi ý các chứng chỉ nên thi, các cuộc thi lớn và các seminar thích hợp để ứng viên có thêm kinh nghiệm thực tế
- **Hệ thống gợi ý hai chiều (Two-way Recommendation):** Không chỉ tìm việc cho ứng viên, hệ thống sẽ phát triển module dành cho Nhà tuyển dụng (HR Portal) để tự động tìm kiếm ứng viên tiềm năng từ kho dữ liệu (CV Pooling).
- **Tối ưu hóa chi phí:** Nghiên cứu áp dụng các mô hình nhỏ hơn (Small Language Models - SLMs) hoặc kỹ thuật lượng tử hóa (Quantization) để có thể chạy một phần tác vụ đơn giản ngay trên thiết bị người dùng (Edge AI), giảm thiểu chi phí gọi API GPT-4.
- **Cơ chế tạo tài khoản và đăng nhập:** Thêm chức năng tạo tài khoản cá nhân và đăng nhập để user có thể Upload và chỉnh sửa hồ sơ cá nhân, kết nối với các user khác
- **Nâng cao khả năng lưu trữ:** tích hợp thêm database (MySQL, Postgresql, ...) để có thể lưu trữ nhiều thông tin hơn như hồ sơ cá nhân, hồ sơ của các công ty tuyển dụng, các bài đăng tuyển dụng, ...
- **Scaling ứng dụng:** Nếu trong tương lai, ứng dụng có nhiều người sử dụng hơn thì nhóm sẽ cân nhắc sử dụng K8S để tăng/giảm lượng server tùy thuộc vào số lượng người sử dụng tại thời điểm đó

